

Watchwise

Sequência recomendada de desenvolvimento — Entity → Repository → Service → Controller

Ordem de implementação baseada nas dependências de chave estrangeira do modelo lógico e nos endpoints do OpenAPI da aplicação. A lógica geral: primeiro as entidades "raiz" sem dependências, depois quem depende só delas, e por último quem depende de várias outras entidades (Log, Curtida) ou agrega tudo (Busca, Resumo).

Fase 1 — Fundação — sem dependências de FK

Usuario	entity → repository → service → service test → controller (registro, login, oauth, me, patch) → controller test
Conteudo	entity → repository → service → service test → controller (/conteudos/referencia) → controller test

Por quê primeiro: quase toda tabela do modelo tem FK para USUARIO ou CONTEUDO. Sem essas duas, nada mais compila de verdade.

Fase 2 — Dependem só de Usuario

Seguidor	repository → service → service test → controller (seguidores, seguindo, seguir) → controller test
SeguePessoa	repository → service → service test → controller (seguir-pessoas/{pessoaTmdbId}) → controller test

Não têm FK própria para outra entidade além de Usuário — dá para implementar logo após a Fase 1 sem bloquear nada. Podem entrar no mesmo módulo de Usuário.

Fase 3 — Dependem de Usuario + Conteudo

Comentario	entity → repository → service → service test → controller (/conteudos/{id}/comentarios, delete) → controller test
Avaliacao	entity → repository → service → service test → controller (/conteudos/{id}/avaliacoes, delete) → controller test
Top5	repository → service → service test → controller (/usuarios/{id}/top5) → controller test

Ordem entre essas três é livre — nenhuma depende da outra.

Fase 4 — Dependem de Usuario (e Conteudo via itens)

Lista	entity → repository → service → service test → controller (CRUD de lista) → controller test
ItemLista	entity → repository → service → service test → controller (/listas/{id}/itens) → controller test

ItemLista depende de Lista já existir, por isso vem logo depois dela e não junto com a Fase 3.

Fase 5 — Dependem de Usuario + Conteudo + Comentario + Avaliacao

Log (Diario)	entity → repository → service → service test → controller (/diario, /usuarios/{id}/diario) → controller test
---------------------	--

LOG tem FK opcional para COMENTARIO e AVALIACAO, então só faz sentido implementar depois que essas duas já existem.

Fase 6 — Depende de Usuario + Comentario + Log

Curtida	entity → repository → service → service test → controller (curtir comentário, curtir log/diário) → controller test
----------------	--

É a entidade mais "tardia" do grafo: precisa de Comentario e Log prontos.

Fase 7 — Satélite — depende de Usuario + Conteudo

Notificacao	entity → repository → service → service test → controller (/notificacoes) → controller test
--------------------	---

Fase 8 — Agregações — sem entidade nova

Resumo	service (agrega Log + Conteudo) → service test → controller (/usuarios/{id}/resumo) → controller test
Busca	service (agrega Lista, Usuario local + proxy TMDB) → service test → controller (/busca) → controller test

Deixe por último porque não são entidades novas — são serviços de leitura que combinam tudo que já foi construído, incluindo a chamada externa ao TMDB.

Por que essa ordem otimiza seu tempo

- Você nunca implementa um repository/service que referencia uma FK para uma entidade que ainda não existe (evita mocks provisórios e retrabalho).
- O módulo Auth sai "de graça" junto com Usuario, já que os endpoints de login/registro só manipulam essa entidade.

- Busca e Resumo ficam por último porque são leitura pura sobre dados que só existem depois que tudo mais estiver populado — testá-los antes seria testar contra um banco vazio.